

Nombre

Willian Ernesto Orellana Montesdeoca

Curso

3er Ciclo

Fecha

11/11/2026

Materia

HERRAMIENTAS PARA EL DESPLIEGUE

Actividad

Promp de ia

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
1	pom.xml	*"¿Qué estructura inicial debe tener el archivo pom.xml para un proyecto de Spring Boot con Kotlin y Java 21?"*	Configurar el bloque <parent>, propiedades de compilación y versión de Kotlin en Maven.	pom.xml, spring-boot-starter-parent, java.version 21, kotlin.version, kotlin-maven-plugin.	Definida la base de Maven, se necesitaron agregar las dependencias web y de base de datos.
2	pom.xml (Dependencias)	*"¿Qué dependencias debo añadir en pom.xml para conectar PostgreSQL, usar Spring Data JPA y ejecutar migraciones con Flyway?"*	Incorporar los starters necesarios para persistencia relacional, controladores REST y control de version	spring-boot-starter-data-jpa, spring-boot-starter-webmvc, spring-boot-starter-flyway, postgresql, jackson-module-kotlin.	Con las dependencias listas, fue necesario configurar los plugins de compi

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
			es de BD.		lación específicos para Kotlin.
3	pom.xml (Plugins Kotlin)	*"¿Para qué sirven los plugins allopen y noarg en el pom.xml de Kotlin con JPA y cómo se configuran?"*	Evitar errores de herencia en clases @Entity y permitir constructores vacíos requeridos por Hibernate.	kotlin-maven-allopen, kotlin-maven-noarg, jakarta.persistence.Entity, MappedSuperclass, Embeddable.	Resuelta la compilación de entidades, se verificó la ejecución del wrapper de Maven.
4	.mvn/wrapper/, mvnw, mvnw.cmd	*"¿Cómo uso el Maven Wrapper (mvnw en Linux/Mac y mvnw.cmd en Windows)*	Asegurar la portabilidad del proyecto en cualquier entorno de	./mvnw spring-boot:run, mvnw.cmd, maven-wrapper.properties, distributionUrl.	Con el entorno de construcción operativo, se procedió a

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		para compilar el proyecto sin instalar Maven globalmente?"	desarrollo.		configurar las propiedades de ejecución de Spring.
5	resources/application.yaml (Servidor)	*"¿Cómo cambio el puerto por defecto de Spring Boot al 8081 y muestro los mensajes de error completos en application.yaml?"*	Evitar conflictos con el puerto 8080 y facilitar la depuración de respuestas HTTP.	server.port: 8081, server.error.include-message: always, spring.application.name.	Configurado el servidor web, el siguiente paso fue enlazar la base de datos PostgreSQL.

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
6	resources/application.yaml (Datasource y JPA)	*"¿Cómo configuro la conexión JDBC a PostgreSQL (base de datos Ejercicio1) y la validación de esquema con Hibernate en application.yaml?"*	Establecer la URL, credenciales, driver de PostgreSQL y modo validate de JPA.	jdbc:postgresql://localhost:5432/Ejercicio1, org.postgresql.Driver, ddl-auto: validate, show-sql: true, format_sql: true.	Al tener ddl-auto: validate, se requirió activar Flyway para que creara las tablas antes de que Hibernate las validara.
7	resources/application.yaml (Flyway)	*"¿Cómo activo y configuro Flyway en application.yaml para que lea los scripts desde*	Habilitar las migraciones automáticas de base de datos al iniciar	spring.flyway.enabled: true, baselineVersion: 1, locations: classpath:db/migration, sqlMigrationPrefix: V,	Con Flyway configurado, se procedió a redact

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		classpath: db/migration?"	la aplicación.	baselineOnMigrate: true.	ar el primer script SQL de creación de tabla.
8	resources/db/migration/V1__init.sql	*"¿Cómo escribo el script V1__init.sql en PostgreSQL para crear la tabla producto con id, description, price y stock?"*	Crear la estructura inicial de la tabla de productos mediante migración versionada.	V1__init.sql, CREATE TABLE producto, primary key, VARCHAR(100), INT.	Posteriormente se requirió estandarizar el nombre de la tabla al inglés mediante una segunda migración.
9	resources/db/migration/V2__RENOMBRAR.sql	*"¿Cómo creo una segunda migración*	Aplicar cambios increm	V2__RENOMBRAR.sql, ALTER TABLE	Tras estructurar la

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		en Flyway (V2__RENOMBRAR.sql) para renombrar la tabla producto a product sin perder datos?"	entales en el esquema de base de datos respetando el historial de Flyway.	producto RENAME TO product;.	primera base de datos, surgió el requerimiento de soportar una segunda fuente de datos en el proyecto.
10	config/PrimaryDbConfig.kt	*"¿Cómo configuro una clase PrimaryDbConfig.kt en Kotlin para definir el DataSource y EntityManagerFactory principal?"*	Aislar la configuración de la base de datos primaria usando anotaciones de Spring.	@Configuration, @Primary, @EnableJpaRepositories, DataSource, LocalContainerEntityManagerFactoryBean.	Definida la base primaria, fue necesario crear la configuración equivalente

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
					para la base secundaria.
11	config/SecondaryDbConfig.kt	*"¿Cómo implemento SecondaryDbConfig.kt en Kotlin para conectar una segunda base de datos sin que choque con la primaria?"*	Gestionar múltiples bases de datos en la carpeta config separando paquetes de entidades y repositorios.	SecondaryDbConfig, @Qualifier, PlatformTransactionManager, separación de basePackages y entityPackages.	Con ambas conexiones configuradas, se procedió a construir la capa de modelos en model/.
12	model/product.kt	*"¿Cómo mapeo la entidad Product en Kotlin dentro de la carpeta model para que*	Representar los registros de productos como objetos	@Entity, @Table(name = "product"), @Id, @GeneratedValue, propiedades id, description, price, stock.	Junto con los productos, el modelo de negoci

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		coincida con la tabla product de PostgreSQL?"	Kotlin gestionados por JPA.		o necesitaba representar órdenes de compra o pedido.
13	model/Order.kt	*"¿Cómo creo la entidad Order.kt en Kotlin con sus anotaciones JPA para gestionar pedidos dentro del paquete model?"*	Definir la estructura de datos y mapeo relacional para la entidad Order.	@Entity, @Table, @Id, tipos de datos Kotlin, serialización JSON.	Con las entidades Product y Order listas, se pasó a crear la capa de acceso a datos en repository/.

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
14	repository/productRepository.kt	*"¿Cómo creo la interfaz productRepository.kt en Kotlin extendiendo JpaRepository para operar sobre Product?"*	Proveer métodos CRUD estándar (findAll, findById, save, deleteById) sin escribir SQL manual.	interface productRepository : JpaRepository<Product, Int>, @Repository.	De igual manera, se necesitó construir el repositorio para interactuar con la tabla de órdenes.
15	repository/OrderRepositor.kt	*"¿Cómo implemento OrderRepositor.kt en Kotlin usando Spring Data JPA para consultar y guardar entidades Order?"*	Habilitar la persistencia de órdenes dentro del paquete repository.	JpaRepository<Order, Int>, @Repository, consultas derivadas de Spring Data.	Con los repositorios operativos, se procedió a crear la capa intermedia de

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
					reglas de negocio en service/.
16	service/productService.kt	*"¿Cómo desarrollo ProductService.kt en Kotlin para implementar las operaciones de listar, obtener por ID, guardar y actualizar parcialmente productos?"*	Encapsular la lógica de negocio de productos y conectar el controlador con productRepository.	@Service, @Autowired, list(), getById(id), save(product), lógica de actualización parcial.	Tras completar el servicio de productos, se replicó la arquitectura para gestionar las reglas de negocio de órdenes.
17	service/OrderService.kt	*"¿Cómo estructuro OrderService.kt en Kotlin para*	Desacoplar el controlador de órdene	@Service, inyección de OrderRepositor,	Con la capa de servicios

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		manejar la lógica de negocio de las órdenes antes de persistirlas?"	s del acceso directo al repositorio y centralizar validaciones.	manejo de flujo de negocio de Order.	lista, el siguiente paso fue exponer los recursos mediante endpoints REST en controller/.
18	controller/productController.kt (GET y POST)	*"¿Cómo implemento en productController.kt los endpoints GET /products, GET /products/{id} y POST /products devolviendo código*	Exponer las operaciones de lectura y creación de productos vía HTTP en formato JSON.	@RestController, @RequestMapping("/products"), @GetMapping, @PostMapping, @ResponseStatus(HttpStatus.CREATED), @RequestBody.	Además de crear y consultar productos, se requirió permitir modificacio

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		201 CREATED?"			nes parciales sobre un producto existente.
19	controller/productController.kt (PATCH)	*"¿Cómo agrego un endpoint @PatchMapping('/{id}') en productController.kt para actualizar parcialmente un producto existente?"*	Permitir actualizaciones parciales mediante el verbo HTTP PATCH pasando el identificador en la ruta.	@PatchMapping("/{id}"), @PathVariable id: Int, @RequestBody product: Product.	Con el controlador de productos completo, se procedió a implementar el controlador de órdenes.
20	controller/OrderController.kt	*"¿Cómo creo OrderController.kt en*	Habilitar la comunicación	@RestController, @RequestMapping, @GetMapping,	Una vez construidas

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		Kotlin para exponer los endpoints REST de creación y consulta de órdenes?"	HTTP/JSON para el recurso de órdenes conectado a OrderService.	@PostMapping, @PathVariable.	todas las capas, se verificó el punto de entrada y las pruebas unitarias del proyecto.
21	TrabajosApplication.kt y TrabajosApplicationTests.kt	*"¿Cómo funciona la clase principal TrabajosApplication.kt y la prueba contextLoads() para verificar que todo el contexto de Spring arranque sin errores?"*	Validar la carga de beans, configuraciones de BD y controladores al iniciar la aplicación.	@SpringBootApplication, runApplication<TrabajosApplication>, @SpringBootTest, @Test fun contextLoads().	Comprobado el funcionamiento local, se preparó el proyecto para subirlo a Git ignorando

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
					archivos innecesarios.
22	.gitignore y .gitattributes	*"¿Qué reglas debo poner en .gitignore y .gitattributes para no subir la carpeta target ni archivos de IntelliJ IDEA a GitHub?"*	Mantener el repositorio limpio de binarios compilados y normalizar saltos de línea de mvnw.	target/, .idea/, *.iml, .vscode/, /mvnw text eol=lf, *.cmd text eol=crlf.	Con las exclusiones configuradas, se inicializó y vinculó el repositorio remoto en GitHub.
23	Git (init, remote, add)	*"¿Cómo inicializo Git en mi proyecto, lo vinculo con mi repositorio de GitHub (willian134/herramien*	Conectar la carpeta local del proyecto con el repositorio remoto	git init, git remote add origin, git status, git add ..	Una vez preparados los archivos en el área de

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
		tas1) y preparo todos los archivos?"	en GitHub.		ensayo (*staging*), se registraron los cambios mediante commits.
24	Git (commit y push)	*"¿Cómo hago commit de mis avances con un mensaje descriptivo y los subo a la rama master en GitHub?"*	Guardar el historial de cambios de la arquitectura multicapa y las reglas de negocio en la nube.	git commit -m "nases de datosy reglas de negocio", git push -u origin master, verificación de los 5 commits.	Finalmente, se revisó la estructura completa en la interfaz web de GitHub para validar que todas las carpet

N.º	Carpeta / Archivo involucrado	Consulta / Prompt utilizado (reconstruido)	Objetivo de la consulta	Información o palabras clave obtenidas	Cómo llevó a la siguiente consulta
					as estuvieran sincronizadas.
25	Repositorio willian134/herramientas1	*"¿Cómo verifico en GitHub que todas las carpetas (config, controller, model, repository, service, resources) se hayan subido correctamente?"*	Confirmar la integridad del repositorio remoto y la visibilidad del código fuente para la entrega.	Navegación de ramas (master), revisión de archivos .kt y .yaml, historial de commits (15f8c7d, af9d3f8).	Cerró el ciclo completo de desarrollo, pruebas, control de versiones y entrega del proyecto.